Muscle Memory for Agents: Compile not Merely Retrieve

Pouya Ghiasnezhad Omran[1], **Soujanya Lanka**[1], **Qin Zhang**[1] **and Tanya Dixit**[1]
[1]Google Cloud FDE

Memory for LLM agents has converged on a single architectural pattern: store experience as text, embeddings, reflections, or rules; retrieve at inference time; let a general-purpose orchestrator interpret what to do. This paper argues that the pattern is the wrong default for personalization. We position Muscle Memory—the practice of compiling recurring user intent into purpose-built specialist agents—as a distinct memory paradigm from retrieval, and we argue that compilation is a better fit for the workloads where current assistants impose a *multi-turn tax* on their users: making them repeatedly correct format, depth, and scope to obtain a domain-appropriate answer.

We support the position with a reference implementation and empirical evidence. The implementation is a four-phase pipeline (Harvest → Analyze → Augment → Evaluate) that mines conversational history, separates behavioral from task patterns, and emits quality-gated executable *compiled specialists* with two-stage trigger matching. On 90 held-out scenarios across five user personas, the augmented assistant wins 32 of 36 cases where a specialist fires, an 88.9% win rate, with a +2.05 personalization gain and only a −0.28 accuracy cost on a 1–4 scale. We discuss why compilation is better suited than retrieval in this regime, what the result implies for the broader memory design space, and what open problems remain.

Keywords: procedural memory, compiled memory, position paper, LLM personalization, multi-agent systems, conversational memory, mini-agent swarms, pattern extraction

arXiv:2608.08995v1 [cs.MA] 10 Aug 2026

1. Introduction

LLM-based assistants have proven to be a flexible and powerful approach to a wide range of tasks, from code generation to business planning. However, a fundamental limitation persists: they treat every user identically and every conversation is bootstrapped. Consider two users of the same assistant. A senior software engineer debugging a web service handler expects a terse, code-heavy response with root cause, fix, and edge cases, all in one turn. A cafe owner asking about quarterly cash-flow projections needs bite-sized, reassuring guidance with no jargon. Both receive the same verbose, generic output and spend three to four additional turns correcting format, depth, and scope before reaching a useful answer for their domain-specific task.

We refer to this recurring cost as the *multi-turn tax*. It is not merely an inconvenience but a structural limitation of current LLM assistants. Users repeat the same corrective instructions session after session because the system never learns that this engineer always wants defensive coding tips, or that this cafe owner gets overwhelmed by long lists.

The position. The dominant response to this problem has been to give the orchestrating LLM more, or better, memory: raw transcripts via retrieval-augmented generation (RAG; Lewis et al., 2020), virtual memory hierarchies (MemGPT; Packer et al., 2024), verbal self-reflections (Reflexion; Shinn et al., 2023), Zettelkasten-style reasoning memory (A-Mem; Xu et al., 2025), or callable skill libraries (Voyager (Wang et al., 2023a), agentic skills (Jiang et al., 2026)). However they differ in what they store, all share a single architectural pattern: *store, retrieve, let the orchestrator interpret*. The orchestrator is asked to translate retrieved memories into action within its own context window

Corresponding author(s): pgomran@google.com

and inference budget.

We argue this default is not enough for personalization. Conversational history should not just be retrieved; it should also be *compiled*—into purpose-built specialist agents, each owning an entire recurring task class, each carrying its own LLM call, prompt, and (where useful) multi-step blueprint, and each tested against real user history before deployment. We call this paradigm Muscle Memory. Where retrieval asks the orchestrator to do more work with more context, compilation removes the orchestrator from the critical path entirely: its role reduces to trigger matching and delegation.

This is analogous to how humans develop *muscle memory*. A pianist learning a sonata initially recruits conscious cognitive control over every finger; with practice, the same passages are committed to motor programs that execute without deliberation. The orchestrator-with-memory paradigm leaves every inference in the conscious-cognition regime. Compiled Memory moves recurring patterns to the muscle-memory regime, where the costly, error-prone step of re-deriving the right behavior from raw context is replaced by a tested, fast, automatic reflex.

The core insight that makes this possible is that follow-up turns in recurring conversations reveal the user's abstract goal: the corrections, clarifications, and pivots are not noise but a signal of what the assistant should have delivered upfront. Muscle Memory extracts these signals and compiles them into compiled specialists that activate at runtime to deliver complete, preference-aligned responses in one or two turns.

We argue that conversational history is an underutilized resource. Rather than storing and retrieving raw fragments, AI systems should extract behavioral patterns and compile them into executable agent code that anticipates user needs proactively. We validate this position empirically on five diverse user personas.

Contributions.

1. A **position**: compiled memory should be treated as a first-class memory paradigm, distinct from retrieval-based approaches, and the right default for personalization workloads.
2. A **reference implementation**: a four-phase pipeline (Harvest → Analyze → Augment → Evaluate) that transforms conversational history into a swarm of compiled specialists (available at[1]).
3. A set of **techniques** that, we argue, recur in any compiled-memory system: *behavioral/task pattern separation* to prevent style cues from triggering task agents; *two-stage trigger matching* combining embedding similarity with per-agent binary questionnaires; *task-adaptive style dampening* that overrides format constraints for complex tasks; and a *three-layer hallucination guard* for cold-start safety.
4. **Empirical evidence** across 90 held-out scenarios on five user personas, achieving an 88.9% win rate with a +2.05 personalization gain—offered as evidence for the position, not merely for the system.

2. Related Work / The Retrieval Paradigm and Its Limits

2.1. Conversational Memory, Reasoning Memory, and Personalization

Endowing LLMs with long-term memory has attracted growing attention in recent years, with methods evolving from raw storage toward increasingly structured memory representations. MemGPT (Packer et al., 2024) introduces a virtual memory hierarchy inspired by operating-system paging, enabling LLMs to manage context beyond fixed windows. LongMem (Wang et al., 2023b) augments language models with a retrieval-based long-term memory that decouples storage from model parameters. At the application level, Persona-Chat (Zhang et al., 2018) grounds dialogue in explicit persona descriptions, while LaMP (Salemi et al., 2024) provides a benchmark for language model personalization across diverse user-facing tasks, and

[1] https://github.com/GoogleCloudPlatform/generative-ai/tree/main/agents/personalized-agent-swarms

PEARL (Mysore et al., 2024) personalizes writing assistants via generation-calibrated retrievers.

More recently, *reasoning memory* systems have moved beyond storing raw text toward capturing problem-solving strategies and procedural knowledge. Reflexion (Shinn et al., 2023) stores verbal self-reflections of task failures and re-injects them on subsequent attempts, achieving reinforcement without weight updates. A-Mem (Xu et al., 2025) uses Zettelkasten-inspired dynamic indexing to create interconnected knowledge networks from experience. Zhang et al. (2025) provide a comprehensive survey of agent memory mechanisms, distinguishing semantic, episodic, and procedural memory, while Wu et al. (2025) formalize agent skills as procedural memory and identify four bottlenecks (discovery, verification, decomposition, scaling) in automated skill synthesis. LATS (Zhou et al., 2024) unifies reasoning, acting, and planning via Monte Carlo Tree Search with LLM-powered value functions, storing search trajectories as experience for future tasks.

Despite these advances, all of the above share a fundamental architectural pattern: they enrich the memory of a *single orchestrating LLM*, which remains responsible for interpreting retrieved memories, reasoning over them, and generating the final output. Whether the memory contains raw text (MemGPT), reflections (Reflexion), or procedural rules (A-Mem), the orchestrator must translate memory into action within its own context window and inference budget. Muscle Memory departs from this paradigm entirely. Rather than retrieving instructions or reasoning traces and feeding them back to a general-purpose orchestrator, Muscle Memory *compiles* recurring patterns into standalone executable mini-agents, each with its own dedicated LLM call, model parameters, enriched prompt, and optional agentic blueprint (e.g., critic/generative loops, multi-step pipelines such as diagnose → recommend → format). The orchestrator does not interpret the memory; it *delegates* the task to a purpose-built agent that has been tested against real conversation history before deployment.

2.2. Skill Libraries and Agentic Skills

A parallel line of work stores reusable *skills* rather than memories. Voyager (Wang et al., 2023a) maintains an ever-growing library of executable JavaScript functions for Minecraft, indexed by embedding similarity and retrieved at task time. In software engineering, SWE-agent (Yang et al., 2024) defines specialized agent-computer interfaces for code editing and navigation, while recent work systematizes agentic skills (Jiang et al., 2026) as callable modules with applicability conditions, execution policies, and termination criteria. Coding assistants such as Claude Code and Cursor allow users to define reusable "skills" or "rules" that are injected into the orchestrator's context when triggered.

These skill-based approaches represent an important step toward executable procedural knowledge. However, they share a critical limitation: retrieved skills are *executed by the orchestrator LLM itself*. The orchestrator receives a skill's instructions or code template and must incorporate them into its own reasoning and generation process. This means the skill's quality is bounded by the orchestrator's ability to follow instructions faithfully, which is particularly challenging for complex, multi-step tasks where format constraints, domain expertise, and quality requirements interact.

Our approach is fundamentally different. Each mini-agent is a *self-contained execution unit* with an axis of freedom from context accumulated so far with the main orchestrator: it makes its own LLM call with a task-specific enriched prompt, can implement multi-step agentic patterns (e.g., a financial-planning agent that drafts → validates assumptions → formats for a non-technical audience), and has been quality-gated through critic passes, fact-checking, and mini-eval validation before deployment. The orchestrator's role reduces to trigger matching and delegation; the complex, quality-critical work is offloaded to a tested, purpose-built agent rather than being handled by the same general-purpose LLM that processes all other requests. This architectural distinction, *delegation to tested specialists* versus *instruction retrieval for a generalist*, is what enables Muscle Memory to achieve an 88.9% win rate on

personalized tasks without sacrificing accuracy.

2.3. Multi-Agent Systems

Recent frameworks orchestrate multiple LLM agents for complex task solving. AutoGen (Wu et al., 2023) enables flexible multi-agent conversations with customizable interaction patterns. MetaGPT (Hong et al., 2024) assigns software-engineering roles (architect, engineer, QA) to agents collaborating via structured standard operating procedures. CAMEL (Li et al., 2023) explores emergent behaviors through communicative agent role-playing, while AgentVerse (Chen et al., 2023) investigates dynamic group composition. Parallel to structured collaboration, recent work explores scaling multi-agent systems via sampling and voting mechanisms (Li et al., 2024). In all these systems, agents are *handcrafted* or *task-specific*: a human designer defines each agent's role, prompt, and tool set. Muscle Memory generates agents *automatically* from user data, with quality gating that determines the optimal number and scope of agents per user (1–6 in practice). Moreover, while existing multi-agent frameworks focus on *decomposing a single complex task* across collaborating agents, Muscle Memory creates agents that each *own an entire task class*, that is, a recurring user need learned from history, and operate autonomously without inter-agent coordination.

2.4. LLM-as-Judge Evaluation

Automated evaluation of LLM outputs increasingly relies on LLMs themselves as judges. MT-Bench and Chatbot Arena (Zheng et al., 2023) establish pairwise comparison as a scalable evaluation paradigm, while AlpacaEval (Dubois et al., 2024) introduces length-controlled automatic evaluation to reduce verbosity bias. Wang et al. (2024) identify position bias in LLM judges and propose debiasing strategies. Most existing benchmarks evaluate *single-turn* outputs against static references. In contrast, Muscle Memory introduces a *user-agent-driven* evaluation protocol where a simulated user autonomously drives multi-turn conversations with both baseline and augmented assistants, providing a more realistic assessment of personalization quality over extended interactions.

3. The Compiled Memory Position

We define Muscle Memory as the practice of converting recurring patterns from a user's interaction history into standalone executable specialist agents—each owning an entire task class, each invoked at runtime via trigger matching rather than prompt augmentation, and each tested before deployment. This section articulates the position as three principles, contrasts compiled memory with retrieved memory across the dimensions where they diverge, and surfaces the assumptions that determine when compilation is the right choice.

3.1. Three principles

P1: Compilation over retrieval. A pattern that recurs across sessions is evidence of a stable workload. Stable workloads should be compiled into code, not re-derived from retrieved fragments at every inference. Retrieval is appropriate for one-off facts whose interpretation depends on the live request; compilation is appropriate for recurring procedures whose interpretation is itself the part that should be cached.

P2: Specialists over generalists. A purpose-built agent with a task-specific enriched prompt, dedicated model parameters, and (optionally) a multi-step blueprint outperforms a generalist orchestrator that has been handed equivalent instructions and asked to follow them. The specialist is bounded by what it has been tested on, not by what an instruction-following generalist happens to attend to in its context window. A specialist also has an axis of freedom from the context accumulated so far with the main orchestrator: it begins each invocation with its own prompt, not the orchestrator's drift.

P3: Tested before deployment. A compiled specialist is an artifact: it can be quality-gated through critic passes, fact-checks, and held-out evaluations *before* it is allowed to run in production. Retrieved-then-interpreted memory cannot

be tested in this way; its behavior emerges only at inference, conditioned on the orchestrator and the rest of the prompt, and is therefore difficult to audit, regression-test, or version.

3.2. Compiled vs. retrieved: where they diverge

Compiled and retrieved memory differ along several dimensions that matter for personalization workloads. Where retrieved memory ships textual context to the orchestrator at every call, compiled memory ships an executable artifact that is invoked instead of the orchestrator. Where retrieval quality is bounded by the orchestrator's instruction-following, compiled quality is bounded by the specialist's own prompt and blueprint, both of which are tested. Where retrieved-memory failures appear at inference and are hard to localize, compiled-memory failures show up as failed quality gates before deployment, or as miss-routes at trigger matching, both of which are debuggable in isolation.

The cost structure also differs. Both paradigms incur per-call overhead, but its nature and scaling differ. Retrieval embeds the query, searches a store, and injects retrieved fragments into the orchestrator's prompt, inflating input tokens on every call by an amount that grows with the breadth of stored experience. Compilation pays a lightweight, fixed-size routing cost (one feature-extraction call plus one embedding lookup; see Section 4.3) that is independent of how many patterns have been compiled. Compilation additionally pays an up-front compile cost amortized over the lifetime of each specialist. The trade-off favors compilation precisely when patterns are stable and recurrence is high, the regime that personalization occupies.

3.3. When compilation is the right choice

Compiled memory is not a universal replacement for retrieval. It is the right default when:

- The same intent recurs frequently enough across sessions to amortize compilation cost.
- Quality, style, or workflow consistency matters across instances of that intent.
- Patterns can be discovered from observed history rather than only being declared by the user up front.

Where these conditions fail—one-shot factual queries, novel tasks with no history, exploratory dialogue—retrieval (or no memory at all) remains the correct choice. The two paradigms are complementary, but the field has defaulted to retrieval even where compilation would dominate.

4. A Reference Implementation

Muscle Memory operates as a sequential four-phase pipeline (Figure 1). Each phase produces artifacts consumed by the next: raw sessions → pattern-derived swarms → augmented runtime → comparative evaluation logs.

4.1. Phase 1: History Harvest

We generate 250 synthetic multi-turn conversations across five user personas spanning diverse domains: a senior software engineer, a marketing manager, an ML graduate student, a small-business (cafe) owner, and a travel/cooking enthusiast. Each persona has 50 predefined scenarios with a specific intent and one of four follow-up strategies (*clarify*, *deep_dive*, *pivot*, *correct*). A dual-LLM orchestration drives each session: a user agent role-playing the persona (temperature 0.9 for diversity) interacts with a standard assistant, producing naturalistic multi-turn dialogues that capture realistic preference-expression patterns.

4.2. Phase 2: Pattern Analysis and Swarm Generation

The analysis pipeline extracts, validates, and compiles mini-agents through eight stages.

Pattern extraction and classification. Sessions are batched (10 at a time) and analyzed by an LLM to identify recurring patterns (frequency ≥ 3), which are merged via Jaccard similarity (> 0.5) to consolidate overlapping intents. Patterns are classified as either *task patterns* (what the user wants: financial planning, code debugging) or

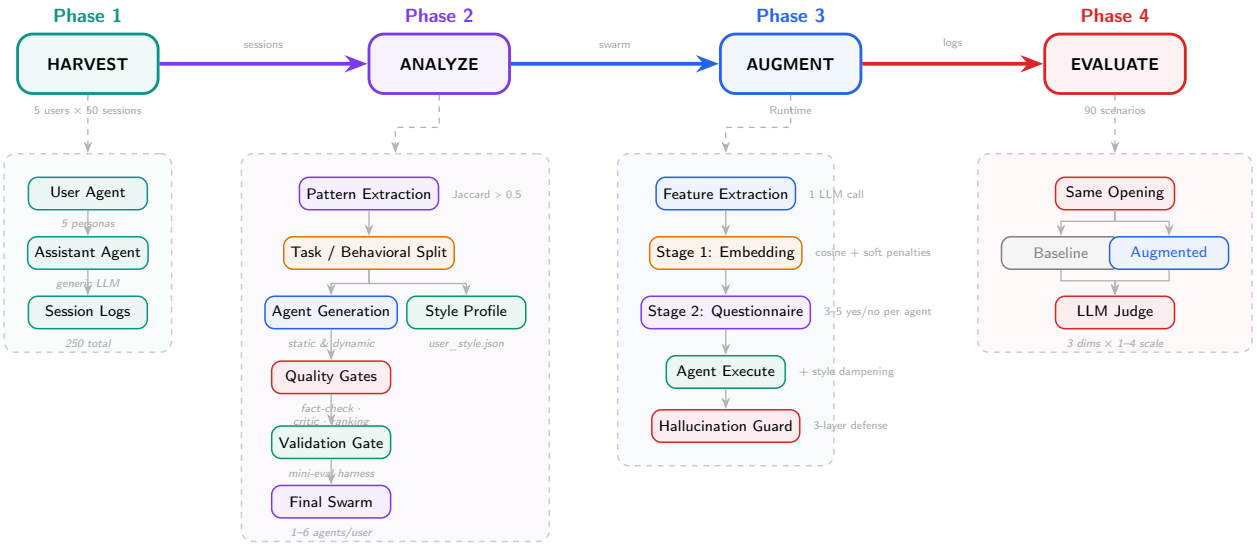


Figure 1 | **Muscle Memory system architecture.** Four sequential phases produce personalized mini-agent swarms from conversational history. **Harvest** generates 250 synthetic multi-turn sessions across 5 user personas. **Analyze** extracts recurring patterns, separates task from behavioral signals, generates executable agents with multi-stage quality gating (fact-check, critic pass, non-parametric ranking, merge, mini-eval validation), and pre-computes scope embeddings and binary questionnaires. **Augment** deploys agents at runtime via two-stage matching (embedding similarity + questionnaire disambiguation) with hallucination guard and task-adaptive style dampening. **Evaluate** runs 90 held-out head-to-head comparisons with an LLM judge.

behavioral patterns (how the user communicates: brevity preference, overwhelm avoidance).

This separation is a key design decision. Treating behavioral patterns identically to task patterns produces agents that trigger on communication style cues rather than actual task intent, leading to high false-positive rates. By separating behavioral patterns into a shared style profile (`user_style.json`) applied to all agents at runtime, these false positives are eliminated entirely.

Agent generation and quality gating. Task patterns are compiled into executable Python modules with enriched prompts containing baked-in user preferences. Agents follow two architectures: *static* (single LLM call with enriched prompt) and *dynamic* (multi-step pipeline, typically 2–3 stages such as diagnose → recommend → format).

To ensure quality, each agent passes through a multi-stage gate:

(a) **Parallel generation**: two candidates generated at temperatures 0.2 and 0.35; a lightweight LLM selects the more grounded version.

(b) **Post-generation fact-check**: scans the enriched prompt for fabricated claims; low-confidence assertions are revised with uncertainty caveats.

(c) **Critic pass**: validates each agent against real conversation history on 10 criteria with a 7/10 threshold; a fabrication hard-ceiling caps the score at 4/10 if ≥ 2 contradictions are found.

(d) **Non-parametric ranking**: scores agents on five weighted dimensions (value $\times 3$, distinctiveness $\times 2$, trigger clarity $\times 2$, quality $\times 2$, frequency $\times 1$); agents scoring $\geq 25/50$ are retained.

(e) **Overlap merge**: pairwise overlap detection (embedding cosine + domain/task-type Jaccard ≥ 0.73) triggers an LLM merge-or-keep decision, preventing cross-domain misrouting among semantically similar agents.

(f) **Mini-eval validation**: a subset of the Phase 4 evaluation harness tests each surviving agent on 6 historical scenarios (3 similar, 3 different); agents with average accuracy < 2.5 or

zero trigger rate are pruned.

Trigger pre-computation. For each agent, the pipeline pre-computes a 768-dimensional scope embedding (text-embedding-005) and generates 3–5 contrastive binary (yes/no) questionnaire items for runtime disambiguation.

4.3. Phase 3: Runtime Augmentation

At inference time, each user message passes through the active memory layer (Figure 2a).

Stage 1: Embedding and soft attribute filter. A single LLM call extracts structured features (domain, task type, specificity, keywords, action-object). The user message is embedded (768-dim) and scored against all agent scope embeddings via cosine similarity, with soft penalties for attribute mismatches (-0.15 for domain, -0.10 for task type). All agents above a threshold (≥ 0.45) pass to Stage 2.

Soft penalties, as opposed to hard binary rejection (exact domain match required), allow cross-domain requests to match relevant agents with reduced but non-zero scores, improving recall without sacrificing precision.

Stage 2: Binary questionnaire disambiguation. When a single agent passes Stage 1, it is selected directly. When multiple candidates survive, each candidate's pre-computed questionnaire (3–5 yes/no questions) is evaluated against the user message in parallel LLM calls. The combined score is 40% embedding + 60% questionnaire match ratio. If one agent achieves ≥ 0.8 match ratio, it is selected. If multiple agents exceed this threshold, the highest combined score wins. If none reach 0.8, a stronger LLM tiebreaker is invoked.

The questionnaire stage is essential for domains with semantically similar agents. For the cafe owner persona, embedding similarity alone could not distinguish five cafe-domain agents (menu pricing vs. supplier management vs. operations); the questionnaire stage resolved all misroutes.

Routing overhead. The per-call routing cost of the two-stage matching pipeline is bounded and predictable. Stage 1 requires exactly one lightweight LLM call for feature extraction (structured JSON output, ~500 tokens) and one embedding API call, both fixed costs independent of swarm size. Stage 2 fires only when multiple agents survive Stage 1; when it does, questionnaire evaluations run in parallel, so wall-clock latency equals that of a single call regardless of the number of candidates. In contrast, a retrieval-based system embeds the query (comparable cost), performs a vector search, and injects retrieved context into the orchestrator's prompt, typically adding 500–2,000 tokens of input to the main LLM call on every invocation. Compilation's routing overhead replaces retrieval's context-injection overhead: a fixed-cost routing stage versus a variable-cost prompt inflation. For users with well-separated agent domains (e.g., user_2, where Stage 2 rarely fires), the routing cost reduces to a single Flash call plus an embedding lookup.

Task-adaptive style dampening. When the matched agent detects a complex task (financial plans, pricing strategies, technical architecture), it automatically overrides conflicting style constraints. For example, a user whose style profile specifies "bite-sized, one-concept-at-a-time" responses would have a comprehensive financial plan fragmented across multiple turns; style dampening preserves tone and language level while allowing complete delivery.

Hallucination guard. Fabrication defense operates at two stages. *Generation-time:* the critic pass scores fabricated references and technical claims as critical failures (score 0); agents that exhibit fabrication in the validation gate are pruned before deployment. *Runtime:* on first-turn messages only (where session history is naturally empty), regex scanning detects fabricated-context markers (e.g. "as we discussed," "your message was cut off") and falls back to the baseline response. In multi-turn conversations, mini-agents receive the full session history and these checks are bypassed, avoiding false suppression of valid personalized references.

Table 1 | User personas and generated mini-agent counts.

User	Persona	Agents	Domain
user_1	Software engineer	1	Python, GCP, Docker
user_2	Marketing manager	6	Copy, metrics, strategy
user_3	ML grad student	6	NLP, training, math
user_4	Cafe owner	5	Finance, ops, suppliers
user_5	Travel + cooking	5	Trips, recipes, culture
Total		**23**	

4.4. Phase 4: Evaluation Protocol

We employ a user-agent-driven evaluation protocol (Figure 2b). For each of 90 held-out scenarios (18 per user: 10 similar to training intents, 8 different), a simulated user agent sends the same opening message to both the baseline and augmented assistants and autonomously drives multi-turn conversations until the goal is reached or a turn cap is hit.

Both agents run under *fair evaluation parity*: neither has access to web search or external memory tools; the only difference is the augmented agent's swarm tool. An LLM judge (Gemini 3.1 Pro) scores both transcripts on three dimensions, each on a 1–4 scale: *accuracy* (factual correctness; fabricated technical claims cap the score at 1), *helpfulness* (goal completion), and *personalization* (alignment with learned preferences). The judge then declares a winner.

5. Empirical Evidence

5.1. Setup

The complete source code for our pipeline is publicly available[2]. Table 1 summarizes the five user personas, their domains, and the number of mini-agents generated by the quality-gated pipeline. The total of 23 agents across 5 users reflects the non-parametric selection: no fixed target is imposed, and only agents scoring $\geq 25/50$ on the ranking rubric survive. Evaluation uses 90 held-out scenarios (50 similar, 40 different) with an LLM judge (Gemini 3.1 Pro, temperature 1.0).

[2] https://github.com/GoogleCloudPlatform/generative-ai/tree/main/agents/personalized-agent-swarms

Table 2 | **Head-to-head results when mini-agents fire.** Accuracy and personalization scores on 1–4 scale for similar-domain scenarios. $B \rightarrow A$ = baseline → augmented. *(Excludes 8 false-positive firings).*

User	Fired	W-L-T	Acc. Δ	Pers. Δ
user_1	5/10	3-2-0	−1.00	+1.20
user_2	10/10	**10-0-0**	0.00	+2.50
user_3	9/10	8-1-0	−0.56	+2.23
user_4	6/10	**6-0-0**	0.00	+2.17
user_5	6/10	5-1-0	0.00	+1.67
All	**36/50**	**32-4-0**	**−0.28**	**+2.05**

Our experiments are designed to validate the following observations:

1. Muscle Memory significantly outperforms the baseline assistant in terms of personalization when mini-agents fire.
2. The accuracy cost of personalization is minimal, indicating that the quality-gating pipeline produces reliable agents.
3. The pipeline components (behavioral separation, embeddings, hallucination guard, questionnaires) are complementary; no single component subsumes the others.

5.2. Main Results

Table 2 presents head-to-head results when mini-agents fire. Across all users, the augmented assistant wins **32 of 36** scenarios where a swarm agent activates, achieving an **88.9% win rate**. Personalization improves by **+2.05** on a 1–4 scale (from 1.67 to 3.72) with a modest accuracy cost of **−0.28** (from 3.92 to 3.64).

Two users are particularly noteworthy. **user_2** (marketing manager) achieves a perfect 10–0 record with zero accuracy loss and +2.50 personalization, the highest individual gain. This user's six agents cover well-separated task domains (A/B testing, metrics analysis, email campaigns, social media, presentations, strategy), making trigger matching unambiguous. **user_4** (cafe owner) achieves a 6W–0L record despite having five semantically similar cafe-domain agents; the binary

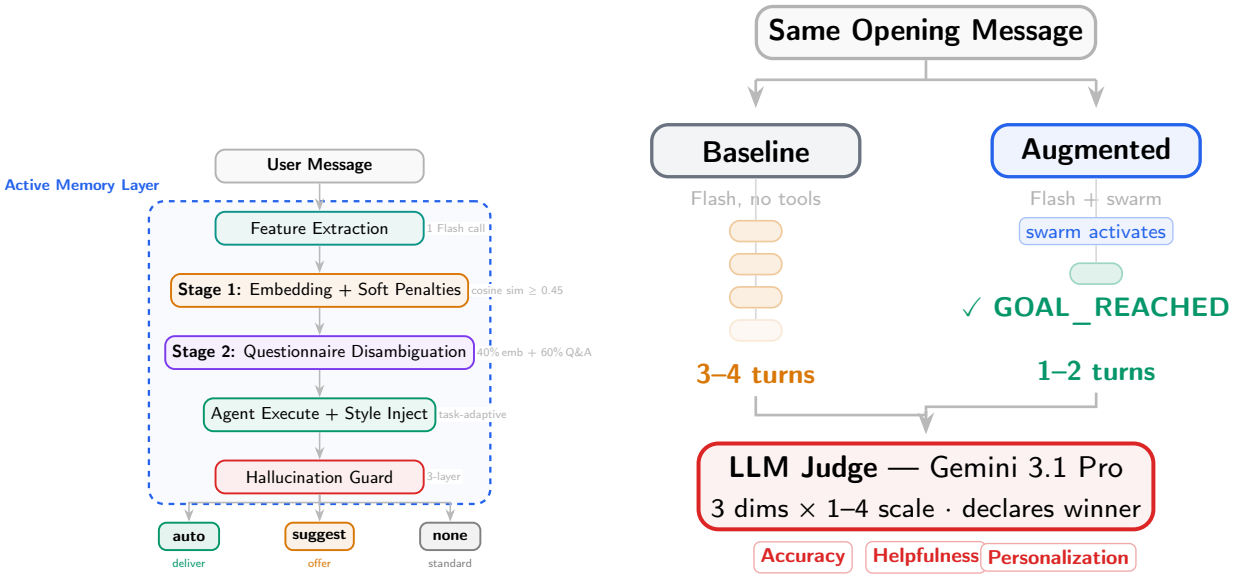


Figure 2 | **(a)** Runtime message flow through the active memory layer with two-stage matching. **(b)** Evaluation protocol: same opening message drives parallel conversations scored by an LLM judge.

questionnaire disambiguation stage successfully distinguishes these overlapping agents.

Trigger accuracy across all users is **72%** (36/50 similar scenarios fire). The false-positive rate is **20%** (8/40 different-domain scenarios trigger an agent). However, these false positives are rarely damaging: the augmented assistant still wins 5 of 8 false-positive cases.

Design lessons. The following observations are qualitative lessons from iterative development, not formal ablations; we report them because each failure mode motivated a specific architectural component. The current pipeline is the result of iterative, failure-driven refinement. Early designs that relied on keyword matching for agent triggering produced poor precision. Treating behavioral patterns as task patterns inflated false-positive rates. Hard binary domain filters prevented legitimate cross-domain matches. Omitting hallucination guards allowed fabricated first-turn context. Each of these failure modes motivated a specific component in the final design: scope embeddings, behavioral/task separation, soft attribute penalties, and the two-stage hallucination defense (generation-time pruning plus runtime guard), respectively.

6. Discussion and Limitations

Where augmentation excels. The strongest results emerge for users with well-separated task domains (user_2: marketing, 6 agents, 10–0) and users who express strong format preferences that a generic assistant consistently violates (user_4: non-technical, needs reassurance, 6–0). The hybrid paradigm, standard conversation for most interactions with seamless mini-agent activation for recurring patterns, reduces multi-turn friction without sacrificing flexibility.

Where it struggles. user_1 (software engineer) was over-pruned to a single agent, limiting coverage. The non-parametric ranking penalized agents whose scope overlapped with general-purpose LLM capabilities ("explain Python errors" is valuable to the user but indistinct from the baseline). The 20% false-positive rate on different-domain scenarios, while rarely damaging, indicates that embedding similarity alone is insufficient for hard negatives; future work could introduce explicit rejection questionnaires.

Runtime guard limitations. The runtime regex layer covers common fabricated-context phrases but is not exhaustive against diverse LLM phrasings. We chose deterministic regex for zero-

latency inference; false-positive risk is bounded by restricting checks to first-turn messages only. A lightweight LLM-based classifier as a complementary layer is a promising direction.

Limitations. We acknowledge several limitations. First, all evaluation is synthetic: simulated user agents and LLM judges, not human participants. While the user-agent-driven protocol and fair evaluation parity provide more rigorous synthetic evaluation than static benchmarks, a human evaluation study is essential future work. Second, the pipeline uses a single model family (Gemini) for generation, matching, and evaluation; cross-model validation would strengthen generalizability claims. Third, agent triggers are static after generation, that is, the system does not adapt agents based on runtime feedback or evolving user preferences. Fourth, there is no cross-user pattern transfer; each user's swarm is generated independently despite potential shared patterns across users with similar roles.

Accuracy-personalization trade-off. The aggregate accuracy cost (−0.28 for +2.05) is concentrated: user_1 incurs −1.00 (single agent, limited coverage), user_3 incurs −0.56 (specialized ML API surface), while user_2, user_4, and user_5 show zero cost—suggesting the deficit correlates with domain technicality rather than architectural isolation.

Error analysis reveals a consistent pattern: mini-agents produce stylistically personalized responses but hallucinate *technical* details. For instance, a statistics agent wrote `mannwhitneyu(..., continuity=True)` instead of the correct `use_continuity`, crashing at runtime (baseline: 4/4, augmented: 1/4). This is distinct from the conversational fabrication the runtime guard targets ("as we discussed"); the generation-time critic catches many such cases, but specialized API surfaces remain challenging. Retrieval-augmented verification of technical claims and user-controllable personalization levels are promising mitigations. We note that the current quality gates target general factual grounding — verifying that claims are not fabricated and that outputs align with user history — but do not validate domain-specific API correctness. Closing this gap likely requires tool-augmented verification (e.g., executing generated code snippets in a sandbox) rather than LLM-only critic passes.

Runtime cost trade-offs. The routing pipeline introduces per-call overhead that we do not claim is zero. Every query pays one Flash feature-extraction call and one embedding lookup. When multiple agents compete, parallel questionnaire calls add wall-clock latency equivalent to a single Flash call. The Pro tiebreaker, reserved for cases where no questionnaire achieves a ≥ 0.8 match ratio, fires infrequently. Importantly, this routing overhead is *architecturally bounded*: it does not grow with the number of compiled agents or the depth of stored history. This contrasts with retrieval-based approaches, where per-call cost grows with the amount of context injected, a property that becomes increasingly costly as memory accumulates over long user histories. A full empirical latency and token-cost comparison between compilation routing and retrieval-based alternatives is a valuable direction for future work; we note that such a comparison must account for retrieval's own per-call costs (embedding, search, context injection), which are routinely omitted from retrieval-system analyses.

Scope of the position. We do not claim that compiled memory subsumes retrieval. We claim that it is currently under-explored relative to its fit for personalization workloads, and that the field has defaulted to retrieval in regimes where compilation would dominate. The right system is plausibly a hybrid: compiled specialists for recurring intents, retrieval for one-off facts, and a router that knows the difference.

7. Conclusion and Future Work / Implications and Open Problems

In this paper, we have presented Muscle Memory, a pipeline that extracts personalized mini-agent swarms from conversational history and demonstrated that this approach achieves an 88.9% win rate with a +2.05 personalization gain at minimal accuracy cost. Our central argument, that conversational history should produce executable agents rather than merely retrievable memories,

is supported by the experimental results across 90 held-out scenarios, where the full pipeline achieves a 32W–4L record.

The behavioral/task separation principle, two-stage trigger matching, and task-adaptive style dampening are general-purpose techniques applicable beyond our specific pipeline. We believe they represent foundational building blocks for personalized multi-agent systems.

There are several interesting directions for future work. We plan to conduct a human evaluation study replacing simulated users and LLM judges with real participants. We also plan to investigate continuous learning, where agents adapt based on runtime feedback rather than remaining static after generation. Cross-user pattern transfer, enabling shared agents for users with similar professional roles and adaptive personalized harness, is another promising direction. Finally, model-agnostic validation across multiple LLM families and per-user adaptive thresholds for the accuracy-personalization trade-off based on domain sensitivity remain open problems.

Acknowledgments

We would like to thank Ashmita Kapoor for her valuable feedback and for reviewing this paper.

Declaration on Generative AI

During the preparation of this work, the author(s) used LLM-based tools in order to: assist with drafting, perform grammar and spell checks, assist with LaTeX formatting and bibliography management. After using these tools, the author(s) reviewed and edited the content as needed and take full responsibility for the publication's content.

References

W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. *CoRR*, abs/2308.10848, 2023. arXiv:2308.10848.

Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto. Length-controlled AlpacaEval: A simple way to debias automatic evaluators. *CoRR*, abs/2404.04475, 2024. arXiv:2404.04475.

S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. Metagpt: Meta programming for A multi-agent collaborative framework. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

Y. Jiang, D. Li, H. Deng, B. Ma, X. Wang, Q. Wang, and G. Yu. SoK: agentic skills — beyond tool use in LLM agents. *CoRR*, abs/2602.20867, 2026. arXiv:2602.20867.

P. S. H. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. CAMEL: communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye. More agents is all you need. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2402.05120.

S. Mysore, Z. Lu, M. Wan, L. Yang, S. Menezes, T. Baghaee, E. Baez, T. Safavi, L. Hong, E. H. Chi, and S. T. Dumais. PEARL: personalizing large language model writing assistants with generation-calibrated retrievers. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.

C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. Memgpt: Towards

llms as operating systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2024. Also presented at NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following.

A. Salemi, S. Mysore, M. Bendersky, and H. Zamani. Lamp: When large language models meet personalization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *CoRR*, abs/2305.16291, 2023a. arXiv:2305.16291.

P. Wang, L. Li, L. Chen, F. Song, B. Lin, Y. Cao, T. Liu, and Z. Sui. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, and F. Wei. Augmenting language models with long-term memory. *CoRR*, abs/2306.07174, 2023b. arXiv:2306.07174.

Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation. *CoRR*, abs/2308.08155, 2023. arXiv:2308.08155.

Y. Wu, K. Zheng, Z. Deng, R. Yuan, Y. He, W. Chai, Y. Song, S. Lyu, Y. Yan, G. Zhang, W. Chen, and J. Fu. Agent skills from the perspective of procedural memory: A survey. *TechRxiv*, 2025.

W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-mem: Agentic memory for LLM agents. *CoRR*, abs/2502.12110, 2025. arXiv:2502.12110.

J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

S. Zhang, E. Dinan, J. Urbanek, A. Szlam, D. Kiela, and J. Weston. Personalizing dialogue agents: I have a dog, do you have pets too? In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2204–2213, 2018.

Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J. Wen. A survey on the memory mechanism of large language model based agents. *ACM Transactions on Information Systems*, 2025. arXiv:2404.13501.

L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y. Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.